STUBBORN

Documentation technique et fonctionnelle

Application e-commerce Symfony

Élément	Valeur
Projet	Stubborn - boutique de sweat-shirts
Framework	Symfony 8.1
Langage	PHP 8.4
Base de données	MySQL 8.4
Paiement	Stripe Checkout - environnement de test
Dépôt GitHub	https://github.com/Damso974/stubborn

Document de livraison - aucune clé secrète ni mot de passe réel n'est inclus.

Sommaire

1. 1. Présentation du projet
2. 2. Objectifs fonctionnels
3. 3. Environnement technique
4. 4. Architecture du projet
5. 5. Installation locale
6. 6. Configuration des variables d’environnement
7. 7. Base de données et données initiales
8. 8. Authentification et rôles
9. 9. Boutique et catalogue
10. 10. Panier et paiement Stripe
11. 11. Back-office administrateur
12. 12. Validation et sécurité
13. 13. Tests automatisés
14. 14. Lancement et utilisation
15. 15. Git et GitHub
16. 16. Captures d’écran

Les captures suivantes illustrent les principales interfaces de l’application Stubborn : accueil, boutique, fiche produit, panier, authentification et back-office administrateur.

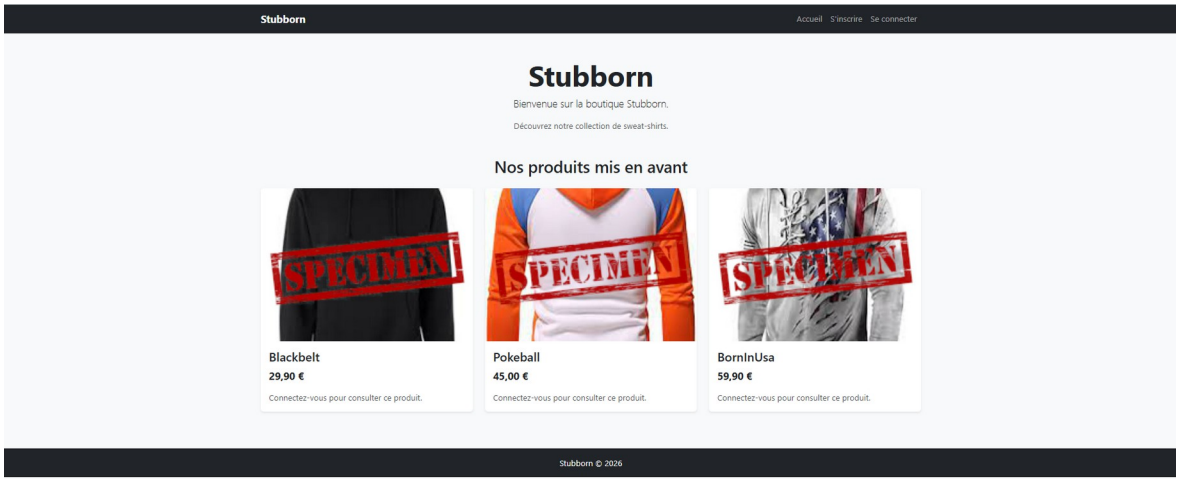


Figure 1 – Page d’accueil (visiteur non connecté).

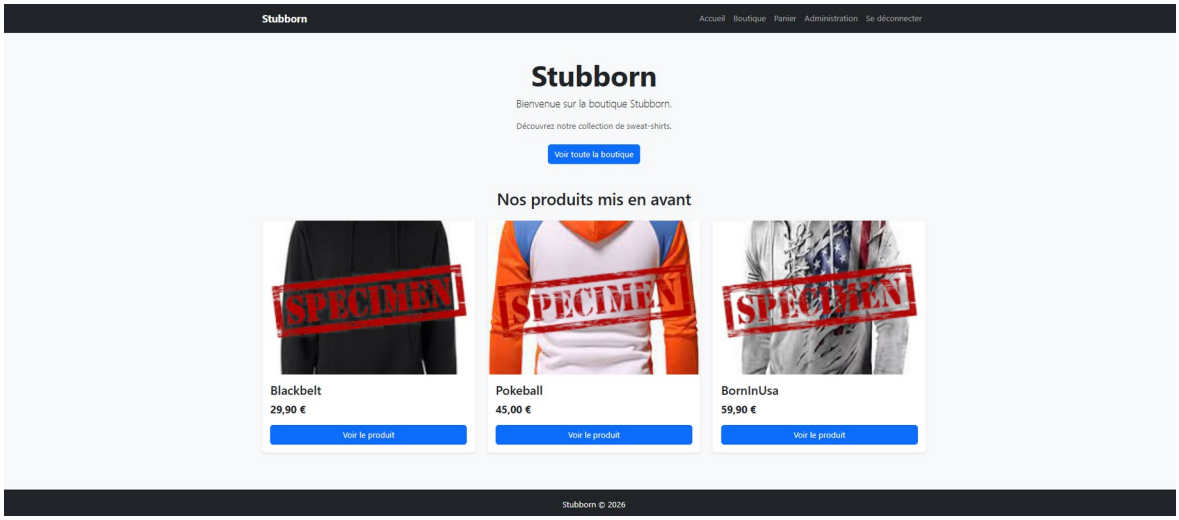


Figure 2 – Page d'accueil (utilisateur connecté).

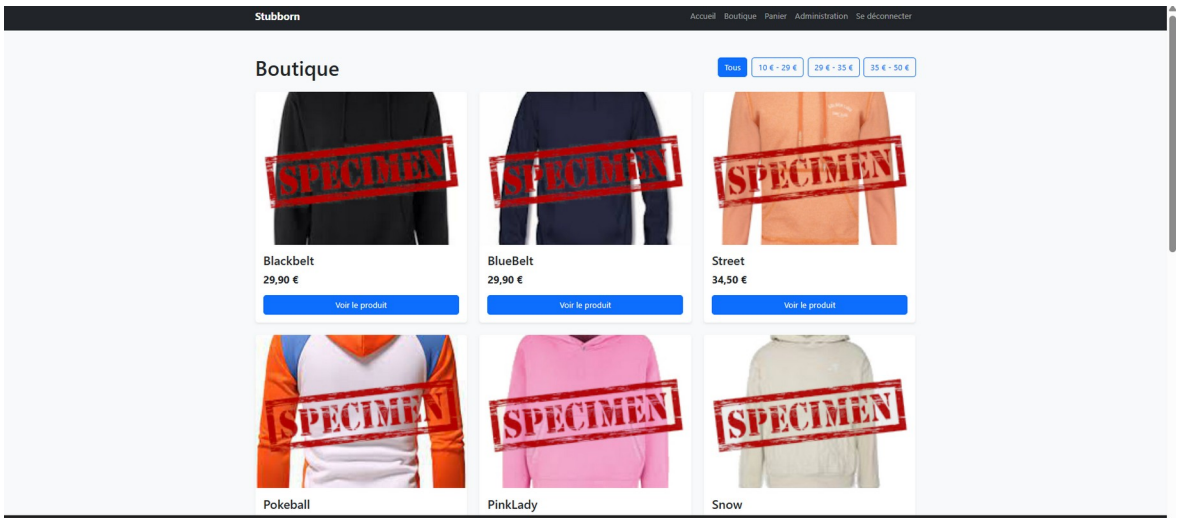


Figure 3 – Boutique : vue principale du catalogue (partie haute).

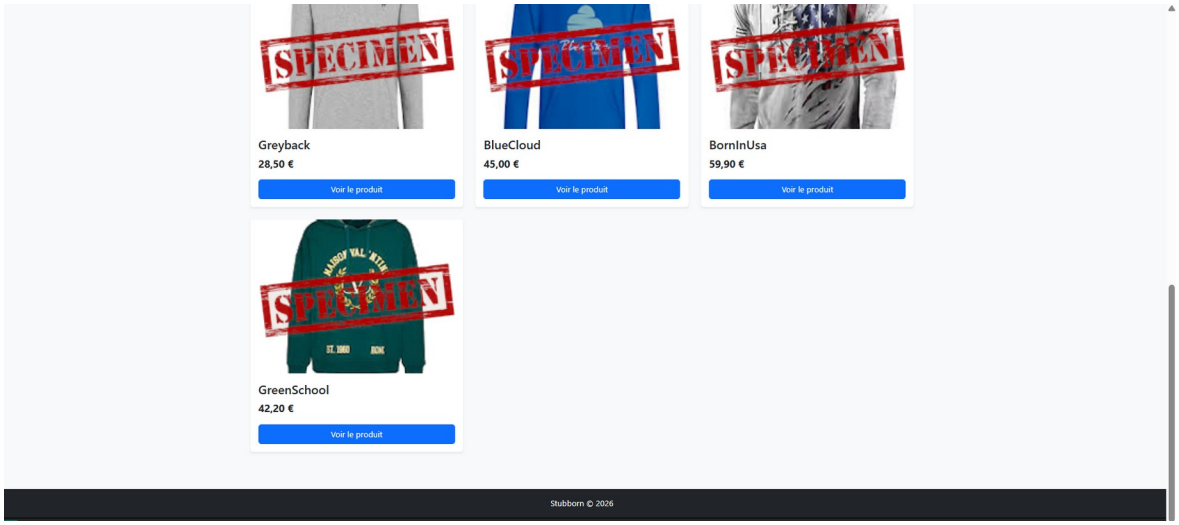


Figure 4 – Boutique : suite du catalogue (partie basse).

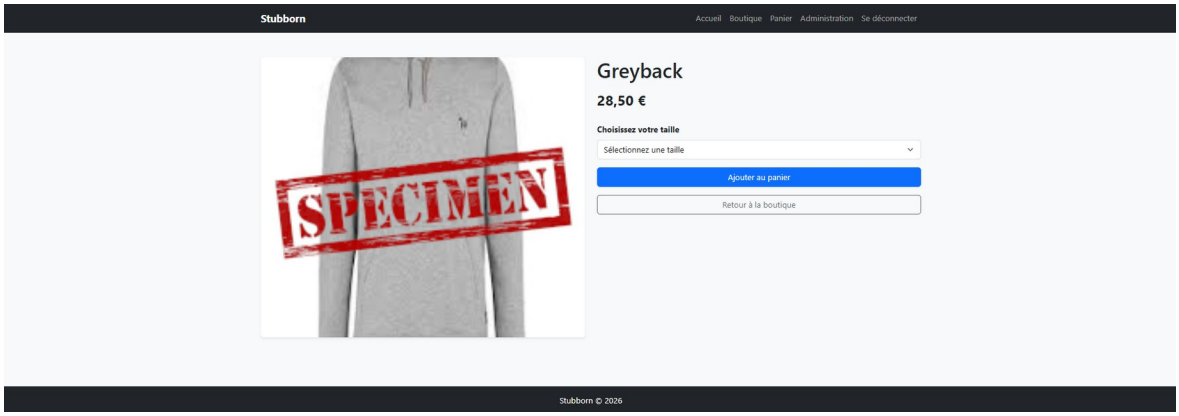


Figure 5 – Fiche produit avec sélection de taille et ajout au panier.

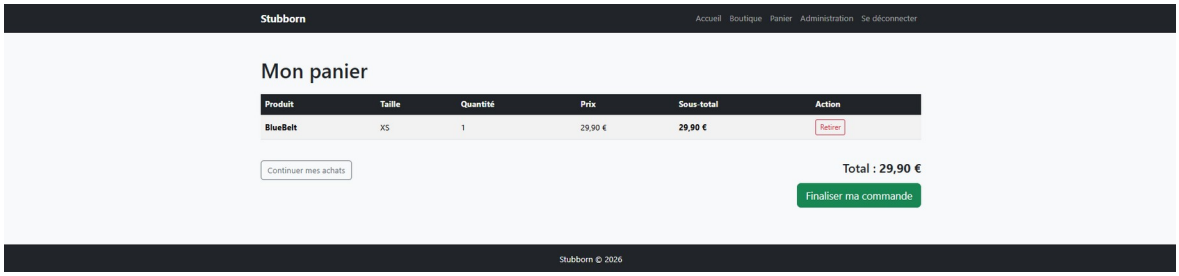


Figure 6 – Panier avec détail des produits, total et validation de commande.

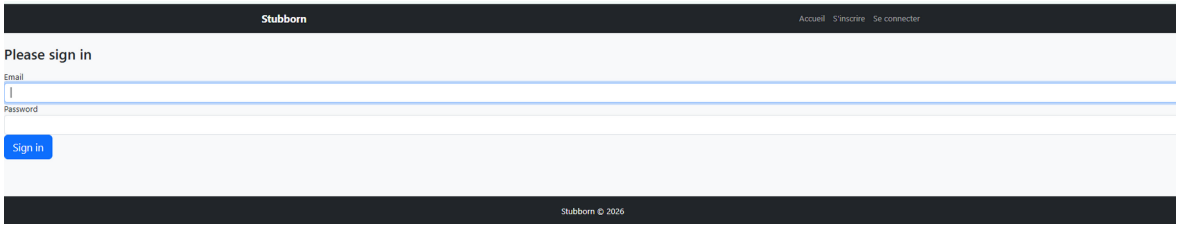


Figure 7 – Écran de connexion.

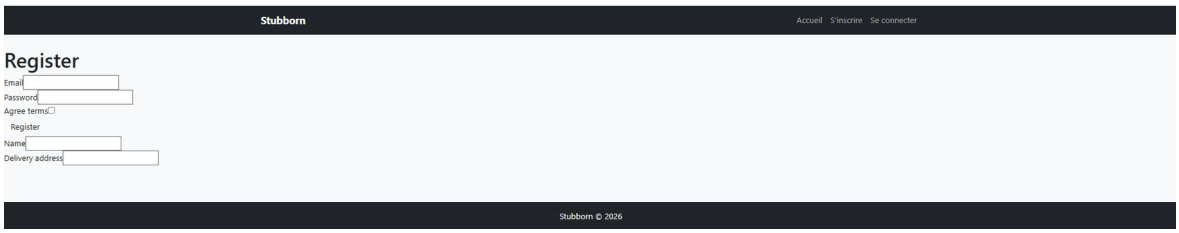


Figure 8 – Formulaire d'inscription utilisateur.

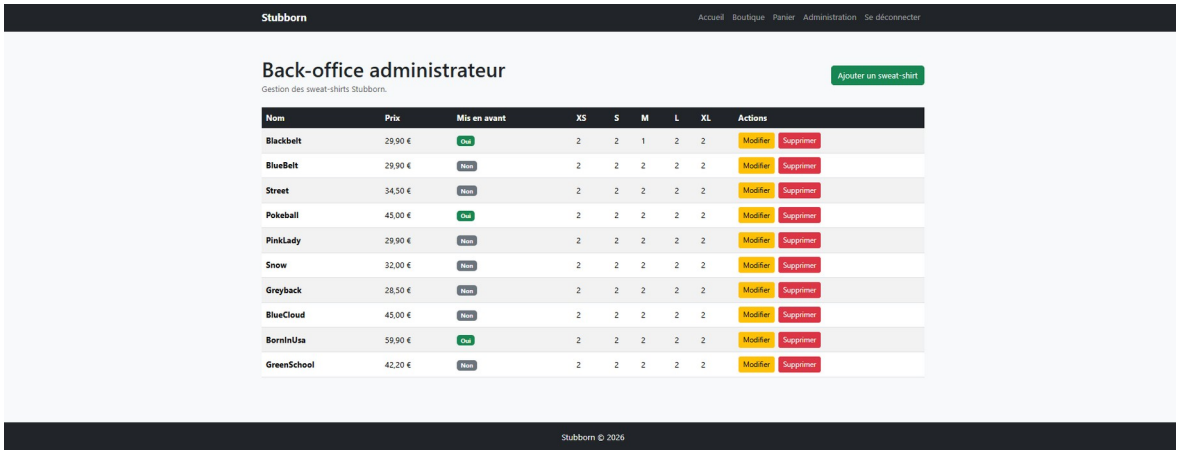


Figure 9 – Back-office administrateur : liste des produits et actions CRUD.

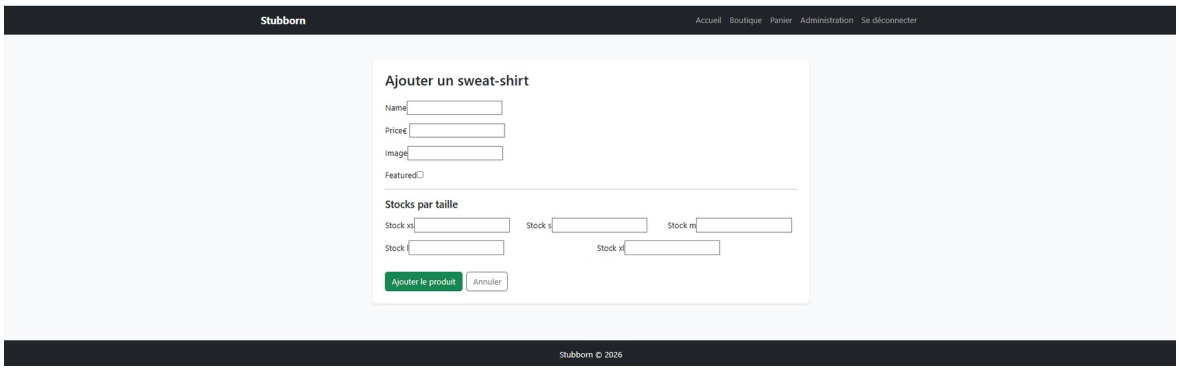


Figure 10 – Back-office : formulaire d'ajout d'un sweat-shirt.

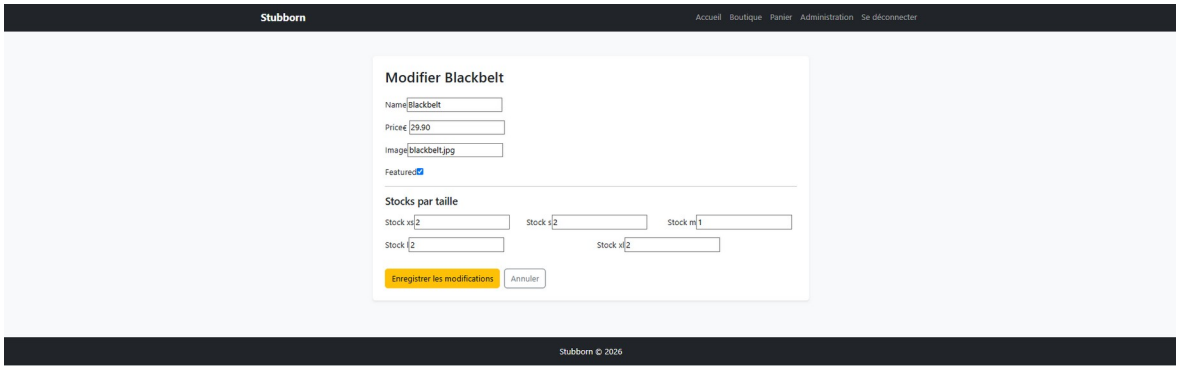


Figure 11 – Back-office : formulaire de modification d'un produit.

17. 17. Bilan

1. Présentation du projet

Stubborn est une application e-commerce de vente de sweat-shirts développée avec Symfony. Elle permet à un visiteur de créer un compte, vérifier son adresse e-mail, consulter les produits, filtrer la boutique par tranche de prix, ajouter un produit au panier en choisissant une taille et effectuer un paiement de test avec Stripe.

Une interface d'administration réservée au rôle ROLE_ADMIN permet de gérer les produits et leurs stocks.

2. Objectifs fonctionnels

- Afficher une page d'accueil avec les produits mis en avant.
- Gérer l'inscription, la connexion, la déconnexion et la vérification de l'adresse e-mail.
- Afficher une boutique de sweat-shirts avec filtres de prix.
- Afficher une fiche produit avec sélection de taille et état du stock.
- Gérer un panier en session : ajout, quantité, retrait et total.
- Créer un paiement Stripe Checkout en environnement de test.
- Décrémenter le stock après confirmation du paiement.
- Fournir un back-office sécurisé avec CRUD complet des produits.
- Tester automatiquement les principales fonctions du panier et du parcours d'achat.

3. Environnement technique

Composant	Version / utilisation
Symfony	8.1.x
PHP	8.4.15
Composer	2.10.x
MySQL	8.4.7 via WAMP
Doctrine ORM	Persistance des entités et migrations
Twig	Templates HTML
Bootstrap	Mise en forme responsive
Stripe PHP SDK	Checkout de test
Mailpit	Réception locale des e-mails
Messenger	Envoi asynchrone des e-mails
PHPUnit	13.4.1
Git / GitHub	Versionnement et dépôt distant

4. Architecture du projet

Le projet suit l'architecture standard Symfony. Les principaux dossiers utilisés sont :

Dossier	Rôle
src/Controller	Contrôleurs HTTP : accueil, boutique, panier, administration, sécurité
src/Entity	Entités Doctrine Product et User
src/Repository	Accès aux données et requêtes spécifiques

src/Form	Formulaires Symfony : inscription et produit
src/Service	Logique métier du panier et intégration Stripe
src/Security	Vérification d’e-mail
src/DataFixtures	Données initiales des produits
templates	Vues Twig
public/images/products	Images du catalogue
migrations	Migrations Doctrine
tests	Tests unitaires et fonctionnels

5. Installation locale

Pré-requis : PHP 8.4, Composer, Symfony CLI, MySQL et Git. Docker est utilisé pour Mailpit dans l'environnement de développement.

5.1 Cloner le dépôt

```
git clone https://github.com/Damso974/stubborn.git
cd stubborn
```

5.2 Installer les dépendances

```
composer install
```

5.3 Configurer les variables locales

Créer ou compléter le fichier .env.local. Ce fichier est ignoré par Git et doit contenir les valeurs propres à la machine locale.

```
DATABASE_URL="mysql://root:@127.0.0.1:3306/stubborn?serverVersion=8.4.7&charset=utf8mb4"
MAILER_DSN="smtp://127.0.0.1:PORT_MAILPIT"
STRIPE_SECRET_KEY="VOTRE_CLE_STRIPE_TEST"
```

Ne jamais versionner une clé Stripe réelle ou une clé de test personnelle.

5.4 Créer la base et appliquer les migrations

```
symfony console doctrine:database:create
symfony console doctrine:migrations:migrate
```

5.5 Charger les données de démonstration

```
symfony console doctrine:fixtures:load
```

Attention : doctrine:fixtures:load sans --append vide les tables gérées avant de recharger les fixtures.

5.6 Démarrer le serveur Symfony

```
symfony server:start
```

Application locale : http://127.0.0.1:8000

6. Configuration des variables d'environnement

Le fichier .env contient uniquement des valeurs génériques. Les paramètres locaux et secrets sont placés dans .env.local.

Variable	Utilité
DATABASE_URL	Connexion MySQL
MAILER_DSN	Serveur SMTP local Mailpit
STRIPE_SECRET_KEY	Clé privée Stripe en environnement de test

Le .gitignore du projet ignore notamment .env.local, .env.*.local, /vendor/ et /var/.

7. Base de données et données initiales

7.1 Entité User

- email : identifiant unique de connexion
- roles : rôles Symfony
- password : mot de passe haché
- name : nom de l'utilisateur
- deliveryAddress : adresse de livraison
- isVerified : statut de vérification e-mail

7.2 Entité Product

- name : nom du sweat-shirt
- price : prix unique toutes tailles
- image : nom du fichier image
- featured : produit mis en avant
- stockXs, stockS, stockM, stockL, stockXl : stocks par taille

7.3 Catalogue initial

Produit	Prix	Mis en avant
Blackbelt	29,90 €	Oui
BlueBelt	29,90 €	Non
Street	34,50 €	Non
Pokeball	45,00 €	Oui
PinkLady	29,90 €	Non
Snow	32,00 €	Non
Greyback	28,50 €	Non
BlueCloud	45,00 €	Non
BornInUsa	59,90 €	Oui
GreenSchool	42,20 €	Non

8. Authentification et rôles

Symfony Security gère l'authentification et les autorisations. Les mots de passe sont hachés et ne sont jamais stockés en clair.

Rôle / état	Accès principal
Visiteur	Accueil, inscription, connexion
ROLE_USER	Accueil, boutique, fiches produits, panier, paiement
ROLE_ADMIN	Tous les accès utilisateur + administration

8.1 Vérification d'e-mail

Après inscription, un e-mail de vérification est envoyé. En développement, les messages sont capturés par Mailpit. Le transport Messenger étant asynchrone, un worker doit être actif pour traiter la file.

```
symfony console messenger:consume async -vv
```

8.2 Création d'un administrateur

Une commande personnalisée permet de créer un administrateur et de hacher son mot de passe correctement :

```
symfony console app:create-admin
```

9. Boutique et catalogue

- Route /products : liste du catalogue.
- Filtres disponibles : 10-29 €, 29-35 €, 35-50 €.
- Route /product/{id} : fiche détaillée.
- Choix des tailles XS, S, M, L et XL.
- Les tailles épuisées sont désactivées dans le formulaire.
- Le service panier vérifie de nouveau le stock côté serveur avant l'ajout.

Les filtres reposent sur une méthode dédiée ProductRepository::findByPriceRange().

10. Panier et paiement Stripe

10.1 Panier en session

CartService stocke le panier dans la session Symfony. Chaque ligne est identifiée par la combinaison identifiant produit + taille.

- Ajout avec contrôle du stock disponible
- Incrémentation de la quantité
- Suppression d'une ligne
- Calcul du sous-total et du total
- Vidage du panier après paiement confirmé

10.2 Stripe Checkout

StripeService crée une session Checkout en euros. La page /cart/checkout redirige vers Stripe avec un code HTTP 303. Le Turbo Symfony est désactivé sur le formulaire de paiement afin de permettre la redirection externe.

10.3 Confirmation du paiement

La route /cart/success reçoit session_id, récupère la session Stripe et vérifie que payment_status vaut paid. Si la confirmation est valide, le stock est décrémenté puis le panier est vidé.

Dans une application de production, il serait recommandé de confirmer les paiements via un webhook Stripe et de créer une entité Order/Commande afin de rendre le traitement idempotent et traçable.

11. Back-office administrateur

Le préfixe /admin est protégé par ROLE_ADMIN. L'interface propose un CRUD complet sur les produits.

Opération	Fonction
Create	Ajouter un sweat-shirt
Read	Lister produits, prix, mise en avant et stocks
Update	Modifier prix, image, mise en avant et stocks
Delete	Supprimer avec requête POST et protection CSRF

12. Validation et sécurité

- Validation Symfony sur Product : nom obligatoire, image obligatoire, prix strictement positif, stocks supérieurs ou égaux à zéro.
- Protection CSRF sur la suppression administrateur.
- Contrôle serveur des tailles autorisées.
- Contrôle du stock avant ajout au panier.
- Vérification du statut paid côté Stripe avant décrément du stock.
- Accès /admin réservé à ROLE_ADMIN.
- .env.local ignoré par Git afin de protéger les secrets.

13. Tests automatisés

La suite de tests finale comporte 8 tests et 32 assertions. Tous les tests passent.

Fichier	Couverture
tests/Service/CartServiceTest.php	Ajout au panier, limite de stock, calcul du total
tests/Controller/CartControllerTest.php	Accès panier, panier vide, succès sans session Stripe, annulation, checkout simulé avec redirection 303

13.1 Lancer tous les tests

```
symfony php bin/phpunit
```

Résultat obtenu : 8 tests, 32 assertions, sans échec. PHPUnit signale quelques dépréciations/notices dans les tests unitaires, non bloquantes pour le fonctionnement du projet.

14. Lancement et utilisation

18. Démarrer MySQL/WAMP.
19. Démarrer Mailpit si la vérification d'e-mail doit être testée.
20. Lancer le worker Messenger dans un terminal séparé.
21. Lancer le serveur Symfony.
22. Ouvrir http://127.0.0.1:8000.
23. Créer un compte ou utiliser un compte local de démonstration.
24. Pour les paiements, utiliser exclusivement le mode test Stripe.

14.1 Commandes de développement utiles

```
symfony server:start
symfony server:stop
symfony console messenger:consume async -vv
symfony php bin/phpunit
symfony console doctrine:migrations:migrate
```

15. Git et GitHub

Le projet est versionné avec Git et synchronisé avec le dépôt GitHub :

```
https://github.com/Damso974/stubborn
```

Le dépôt local suit la branche origin/master. Avant chaque push, il est recommandé de vérifier :

```
git status
git check-ignore .env.local
git grep "sk_test_"
```

Le fichier .env.local ne doit jamais être commit. Les clés Stripe doivent être régénérées si elles ont été exposées accidentellement.

16. Captures d'écran à ajouter

Pour compléter le rapport de rendu, ajouter des captures représentatives dans cette section ou en annexe :

Capture recommandée	Contenu
Accueil visiteur	Produits featured sans bouton Voir
Accueil connecté	Produits featured avec bouton Voir
Boutique	Catalogue et filtres de prix
Fiche produit	Image, prix, tailles et stocks
Panier	Tableau, total et bouton de paiement
Stripe test	Écran Checkout de test ou confirmation
Administration	Liste produits et actions CRUD
Formulaire admin	Ajout ou modification d'un produit
Tests PHPUnit	Terminal montrant 8 tests / 32 assertions
GitHub	Dépôt Stubborn à jour

17. Bilan

Le projet Stubborn couvre le parcours principal attendu d'une boutique e-commerce : authentification, catalogue, filtres, fiche produit, gestion du panier, paiement Stripe en mode test, mise à jour des stocks et administration sécurisée.

Les prochaines améliorations possibles seraient la création d'une entité Commande, la gestion des webhooks Stripe, l'historique des achats, la gestion d'images uploadées, l'envoi d'e-mails transactionnels réels et un déploiement sur un hébergeur compatible Symfony/MySQL.